UNIVERSITÀ
DI TRENTO

Delivery Autonomus Agents

Autonomous Software Agents
a.y. 2025/2026

Davide Donà - 264467
davide.dona-1@studenti.unitn.it
Andrea Blushi - 264468
andrea.blushi@studenti.unitn.it

June 2026

Contents

1 Problem Statement

The `Deliveroo.js` environment is a grid-based delivery simulation where autonomous agents collect and deliver scattered parcels to designated tiles to maximise their accumulated score. The environment introduces various challenges: agents only perceive tiles within a limited sensor radius at any given time step. Parcel rewards decay at a fixed server-configured interval, creating time pressure that incentivises early collection. Multiple agent types may share the map; agents belonging to the same team cooperate to jointly maximise reward, whereas agents from different teams compete for the same parcels. Additionally, movable crates often obstruct navigable paths, meaning agents must actively push them to clear blocked routes.

We implement a BDI (Belief-Desire-Intention) agent augmented with an optional Large Language Model (LLM) based control layer. The BDI core handles reactive parcel collection, exploration, and crate-clearing autonomously; the LLM layer intercepts high-level instructions, adapts agent behaviour through a set of structured tools, and coordinates multi-agent strategy via periodic team assignments.

2 BDI Agent

Each `sensing` event from the server drives one turn of the standard BDI cycle: *perception* integrates the new observations into the belief state via a Belief Revision process (Section 2.1); *deliberation* regenerates and scores desires, rebuilding the ranked desire queue whose head is the agent's active intention (Section 2.2); *planning* translates that intention into typed, executable steps (Section 2.3); and *execution* emits the corresponding socket actions. Two choices keep the agent responsive: belief-modifying actions update beliefs immediately rather than waiting for the next sensing event, and the executor fires one action per server clock tick instead of blocking on an acknowledgment.

2.1 Belief Revision

The agent's belief state relies on two foundational utility data structures to manage temporal observations, which in turn feed into three specialized sub-systems: `AgentBeliefs`, `MapBeliefs`, `ParcelBeliefs`.

Tracker `Tracker` stores a single latest observation $\langle v, t_{seen} \rangle$ per entity `e`. On each sensing report, `update(e, v)` overwrites the existing entry. The confidence of a tracked observation decays exponentially over time:

$$c(t) = \exp\left(-\frac{t - t_{seen}}{\tau}\right) \tag{1}$$

where τ is the exponential time constant (the duration after which confidence drops significantly).

When the current sensing window covers an entity's last-known position, but the entity is absent from the report, `invalidateAtSensedPositions` nullifies that position without removing the record entirely.

Memory While the tracker maintains only the latest state, `Memory` is a supplementary store that maintains a bounded temporal history of observations per entity, automatically evicting entries older than a defined time threshold.

2.1.1 Agent beliefs

`AgentBeliefs` stores the latest state of every observed agent via `Tracker`, and a bounded position history for each enemy via `Memory`. It also maintains a spatial heat field over the map, aggregating enemy presence across all tracked observations.

2.1.2 Map beliefs

`MapBeliefs` stores the static tile layout, a set of transient blocked tiles each with a configurable TTL, and soft traversal penalties on individual tiles.

Upon map receipt, it identifies and permanently seals *sink tiles*: tiles that an agent can enter but from which no valid pickup-and-deliver loop exists. This situation arises on maps where directional conveyor tiles create one-way corridors: an agent may be carried into a region from which it can never return to a spawn tile, or from which no delivery tile is reachable.

To detect these dead zones, the map is modelled as a directed walkability graph where conveyors enforce one-way edges. Tarjan's algorithm is then applied to find all Strongly Connected Components (SCCs). An SCC is considered *safe* only if it contains at least one spawn tile *and* at least one delivery tile (i.e. it supports the full pickup-and-deliver cycle). All walkable tiles outside a safe SCC are reclassified as walls, preventing the planner from ever routing the agent into an unrecoverable region.

2.1.3 Parcel beliefs

This component tracks the availability and value of delivery targets. Parcels within the current sensing window receive their authoritative reward from the server. Once out of sight, parcel rewards decay linearly at -1 per server-defined interval Δ_t. Each parcel maintains an independent decay clock to prevent drift accumulation. Given a parcel last observed at time t_0 with reward r_0, its estimated reward at time t is:

$$r(t) = r_0 - \left\lfloor \frac{t - t_0}{\Delta_t} \right\rfloor . \tag{2}$$

When $r(t) \leq 0$, the parcel is considered expired and is deleted from the agent's beliefs.

2.1.4 Crate beliefs

`CrateBeliefs` uses the same `Tracker` utility described above to store the latest known position for each crate. On every sensing update, observed crate positions are registered and `invalidateAtSensedPositions` nullifies any entry whose tile falls within the current sensing window but is absent from the report, reflecting that the crate has been pushed.

2.2 Desires and Intention Selection

For the BDI layer, we define a fixed set of 3 core desire types, each parameterized by a target tile:

- **ReachParcel**: navigate to and collect a visible parcel;
- **DeliverParcel**: navigate to a delivery tile while carrying parcels;
- **Explore**: navigate to an unobserved spawn tile when idle (i.e., no reachable parcels or deliveries).

2.2.1 Queue reconstruction

At each deliberation step, the desire queue is fully reconstructed from the current belief state; its head is the committed intention the planner acts on. The queue is sorted by priority first, then by descending score within each desire type. Both `ReachParcel` and `DeliverParcel` desires have a priority of 1, while `Explore` has a lower priority of 0. This ensures that the agent always prefers tangible parcel-related goals over exploration when any are available.

2.2.2 Dynamic Scoring

The agent computes a dynamic score for each desire to guide intention selection. The score is a real-valued estimate of the desire's expected utility, incorporating factors such as the parcel's reward, the distance to the target, and the presence of competitors.

Distance Penalty Each desire's score is inversely proportional to the estimated distance to its target, calculated using the agent's current beliefs. The agent computes the shortest path distance d from its current position to all other tiles using a Breadth-First Search (BFS). We will refer to this distance as d throughout the rest of the section. (Note: when $d = 0$, we assign an infinite score).

ReachParcel Let r_p denote the current estimated reward of a target parcel p, and C the set of parcels the agent is currently carrying. The score S for a `ReachParcel` desire targeting parcel p is calculated as:

$$S = c_p \cdot \frac{r_p + \sum_{c \in C} r_c}{d} \cdot \gamma$$

where c_p is the confidence in the target parcel's last known position (Eq. 1, with $\tau = \tau_c$), reflecting how stale the parcel observation is. A parcel that was just sensed has $c_p \approx 1$ and is scored at full value; one that has been out of sight for several half-lives is discounted accordingly.

γ is a race factor that de-prioritizes parcels likely to be collected by competitors. It is computed as:

$$\gamma = \min\left(1, \frac{d_{\text{enemy}}}{d}\right)$$

where d_{enemy} is the distance from the nearest tracked enemy to the parcel.

Incorporating the total reward of currently carried parcels ($\sum_{c \in C} r_c$) into the score allows the agent to evaluate the utility of picking up a new parcel against the potential value of delivering existing ones. This effectively balances the agent's acquisition and delivery goals.

DeliverParcel For a `DeliverParcel` desire, the objective is to secure the rewards of the parcels currently held by the agent. The score S is computed as the total estimated reward of all carried parcels C, inversely scaled by the shortest path distance d to the delivery tile:

$$S = \frac{\sum_{c \in C} r_c}{d}$$

Explore The `Explore` desire serves as a fallback to locate newly spawned parcels. We first define the normalized observation age τ:

$$\tau = \min\left(1, \frac{\Delta t}{T_{\text{spawn}}}\right)$$

where Δt is the time since the target spawn tile x was last observed, and T_{spawn} is the environment's spawn interval.

The cluster weight w quantifies the structural density of the target's surrounding spawn area. It is computed as the sum of inverse distances to all neighbor spawn tiles within the observation range R_{obs}:

$$w = \sum_{y \in OST} \frac{1}{\delta(x, y) + 1}$$

where OST is the set of spawn tiles such that their Manhattan distance $\delta(x, y) \leq R_{\text{obs}}$.

The enemy heat h actively steers the agent away from sectors currently crowded by competitors. It is accumulated from the history of tracked enemy observations E, combining spatial and temporal decay:

$$h = \sum_{e \in E} \exp\left(-\frac{d_e^2}{2\sigma^2}\right) \exp\left(-\frac{\Delta t_e}{\tau_h}\right)$$

where d_e is the spatial distance to the past enemy observation, Δt_e is the observation's age, σ controls the spatial spread, and τ_h is the temporal decay rate.

Finally, the score for an `Explore` desire is computed as:

$$S = \frac{w \cdot \tau}{d \cdot (1 + h)}$$

2.3 Planning: A* Navigation and Collision Handling

For pathfinder, the agent employs a standard A* search algorithm with a Manhattan distance heuristic, which is admissible in our grid-based environment. The planner incorporates dynamic walkability checks based on the agent's current beliefs, treating temporarily blocked tiles as impassable. Statically, walls and out-of-bounds tiles are always rejected; conveyor tiles are additionally direction-sensitive, blocking movement against their flow.

Plans generated by the A* algorithm are defined as sequences of typed actions: directional `move` steps followed by a terminal `pickup` or `putdown` step. An existing plan is reused across deliberation steps as long as the active desire type and target remain unchanged, and all remaining steps successfully pass walkability re-validation.

Collision Handling Prior to executing a move step, the agent verifies the availability of the target tile. A tile is considered occupied, and thus temporarily blocked, under two conditions: either an enemy is currently observed on it, or an enemy's predicted next position matches it with sufficient confidence.

When the next planned tile is occupied, the collision manager applies a three-stage escalation:

1. **Detour**: replan with the blocked tile treated as a wall. This is accepted only if the detour adds at most a predefined threshold of Δ_{th} steps over the remaining plan.
2. **Wait**: if no acceptable detour exists, pause execution for a random duration drawn from a predefined uniform distribution.
3. **Block and replan**: after the wait expires, or after a specified limit L of repeated invalidations, mark the tile as temporarily blocked for a randomized time-to-live (TTL) duration and completely replan toward the same goal.

This escalation applies only to enemies; teammate collisions are resolved deterministically by agent ID, with the lower ID conceding.

3 PDDL-Based Crate Navigation

Some maps in the Deliveroo.js environment contain movable crates that can obstruct navigable paths. An agent may push a crate one tile in any cardinal direction, subject to the constraints outlined in Section 2.3. We model this crate manipulation problem using PDDL (Planning Domain Definition Language), invoking it as a fallback mechanism when the standard A* planner fails to find an unobstructed route.

3.1 Domain Overview

The `deliveroo-crates` domain is formulated in STRIPS and relies on two primary object types: `tile` and `crate`. The complete state required for crate-aware navigation is captured by the following predicate set:

- `(at ?t)`: The agent is located on tile t.
- `(adj-up/down/left/right ?from ?to)`: Defines the valid directed spatial relationships between adjacent tiles for movement or pushing.
- `(crate-at ?c ?t)`: Crate c occupies tile t.
- `(crate-free ?t)`: Tile t contains no crate, making it available for entry or as a push destination.
- `(crate-space ?t)`: Tile t can structurally support a crate (e.g., it is not a wall or a conveyor belt).

The domain defines eight parameterized actions: four directional *move* actions and four directional *push* actions.

- A move action allows the agent to step into an adjacent `crate-free` tile.
- A push action allows the agent to displace a crate one tile in the specified direction, requiring two preconditions:
  (i) the agent is positioned adjacent to the crate on the opposite side of the push direction, and

(ii) the target destination is a valid `crate-space` and is currently `crate-free`.

The execution of a push action simultaneously relocates the crate to the target tile and moves the agent onto the crate's former tile, updating the `crate-free` state of the involved tiles accordingly.

3.2 Planning With PDDL

PDDL planning is invoked exclusively when standard A* pathfinding fails and the agent's belief system indicates that crates obstruct the route (Section 2.3). This detection occurs in two phases. First, an A* search is executed, treating all crate-occupied tiles as impassable. If this search fails, a second A* search is performed that entirely ignores crate positions, yielding a structurally valid but potentially obstructed path. Any crates from the agent's current beliefs that intersect this path are collected; their presence confirms that the route is blocked by movable obstacles rather than static map geometry.

A PDDL problem instance is subsequently constructed from the agent's current beliefs. The `:init` state is populated with the following:

- `(at t_X_Y)`, where (X, Y) is the agent's current position;
- adjacency facts for every pair of connected non-wall tiles, covering all four cardinal directions;
- `(crate-at ?c ?t)` and `(crate-free ?t)` facts for each believed crate position;
- `(crate-space ?t)` facts for all structurally valid positions (non-wall, non-conveyor tiles).

The `:goal` section comprises a single ground atom, `(at t_TX_TY)`, where (TX, TY) represents the target position. Consequently, the solver is tasked with generating a *complete* sequence of `move` and `push` actions from the agent's current location to the final destination, clearing any obstructing crates along the way. The resulting plan is translated back into the agent's internal action representation, stored in the plan buffer, and executed step-by-step until completion or failure.

However, this formulation suffers from a fundamental performance bottleneck: the PDDL solver is slow for the interactive demands of the environment. An initial optimization involved transitioning to a local solver, which eliminated the overhead of remote API requests and constant problem serialization. A more structural optimization, caching solved plans, is unfortunately undermined by the formulation itself. Because the cache key must incorporate both the start and goal tiles, even a trivial change in the agent's position alters the key and triggers a cache miss, despite the crate-clearing maneuver remaining identical regardless of where the agent initiated the request.

3.3 A Smart Solution: Problem Restatement & Caching

The root cause of this inefficiency is that PDDL is only strictly necessary for navigating the crate cluster itself; the initial and final segments can be efficiently handled by standard A* navigation.

To resolve this, the problem is reformulated so that the PDDL solver is only responsible for the segment of the journey that requires crate manipulation.

The overall plan is decomposed into three distinct segments:

- Move from the current position to the cluster entry tile (A*);
- Navigate through the crate cluster, from entry to exit (PDDL);
- Move from the cluster exit tile to the final goal (A*).

Under this paradigm, the PDDL problem is anchored to the cluster's entry tile rather than the agent's live position. Computed maneuvers are stored in a FIFO cache, utilizing the tuple (entry, exit, {crate positions}) as the key.

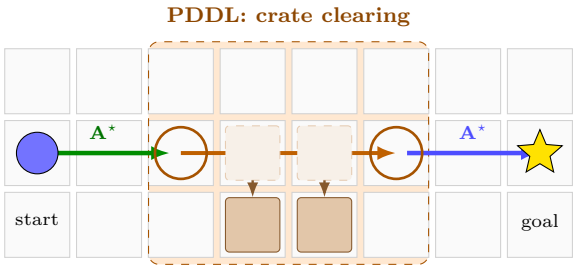

Figure 1: Three-segment path decomposition. The agent uses A* for the initial approach and final exit, restricting computationally expensive PDDL crate-clearing to the localized cluster. This isolation enables efficient, position-invariant plan caching.

4 LLM-Based Agent Layer

The LLM layer functions as a thin wrapper around the underlying BDI agent. It interprets high-level textual commands from a mission controller, utilizing tools exposed by the BDI core to inject new desires, adjust scoring rules, or initiate team coordination.

The mission controller can issue requests that fall into three broad categories: direct navigation goals, persistent behavioural adjustments, and team-level control. Each category is handled through a different mechanism in the BDI core, chosen to integrate naturally with the existing deliberation loop rather than bypassing it.

4.1 Adapting Behavior Through Goal Injection

Navigation requests, more specifically, moving to a tile (*go-to*) or delivering at a specific location (*deliver-at*), are translated into `REACH_TILE` and `DELIVER_PARCEL` desires, respectively. These injected goals enter the BDI desire queue and persist for a specified Time-To-Live (TTL), allowing them to be continuously re-evaluated against existing tasks until completion or expiry.

Both injected desires utilize the standard BDI rate formulas (Section 2.2), but incorporate configurable rewards from the mission controller to dictate their competitive priority:

- **`REACH_TILE`:** Scored identically to `ReachParcel`:

$$S = \frac{r_{\text{inj}} + \sum_{c \in C} r_c}{d}$$

where r_{inj} is the injected reward. Setting this reward equal to a typical parcel value allows the navigation goal to compete on equal footing with standard parcel collection.

- `DELIVER_PARCEL`: Similarly, applies the standard rule-adjusted scoring for deliveries (Section 4.2), including an optional additive bonus after the rule combination to successfully bias the agent toward the requested destination tile.

4.2 Adapting Strategy via Rule Injection

The `RuleStore` maintains a persistent registry of LLM-defined scoring rules. Each rule encodes a state condition alongside a linear effect: a multiplier m and an additive offset a.

During each deliberation cycle, any matching rules are applied to the baseline scores from Section 2.2 as a linear combination. This scales or shifts the priorities without overwriting the foundational scoring logic.

These rules evaluate three primary state variables: the individual parcel reward, the current carry count, and the target delivery tile. Their effects compose directly onto the reward aggregates utilized in the desire formulas.

ReachParcel: The baseline score (Section 2.2) is linearly adjusted according to the *estimated stack count* post-pickup: the hypothetical carry count $|C| + 1$. This allows a rule to incentivize picking up a specific parcel only if it achieves a desired total stack size, leaving all other pickup valuations unchanged.

DeliverParcel: The delivery score is linearly modified based on the current carry count $|C|$ and, optionally, the specific target tile. A stack rule keyed to $|C|$ dynamically alters the attractiveness of delivery depending on the agent's current payload, while a tile-specific rule can shepherd the agent toward a preferred drop-off location. An independent bonus can also be applied directly via the `request_putdown_at` tool, biasing delivery toward a specific tile regardless of any active rules.

Explore: In the absence of an active stack rule, `Explore` maintains its default priority of 0 and functions as outlined in Section 2.2.

However, when a positive stack rule is triggered, `Explore` is elevated to priority 1, forcing it to compete directly against active parcel and delivery goals. Its score is subsequently scaled by the anticipated reward of completing the stack, incentivizing the agent to actively search for missing parcels rather than settling for a premature delivery. This behavior emerges naturally from the linear weighting system: the LLM is not required to issue an explicit exploration command, as simply registering a stack rule is sufficient to trigger the shift.

Traversal Penalties: Traversal penalties operate at the planning layer rather than the deliberation layer. Unlike scoring rules, which modulate desire priority during deliberation, a traversal penalty injects an additive cost onto the A* edge leading onto a specific tile. When the planner evaluates candidate paths, it adds this cost to any step landing on a penalized tile, causing the agent to prefer a detour whenever the detour length is shorter

than the penalty, without completely forbidding the option of traversing the tile if it becomes necessary.

4.3 Coordination

Coordination is fully decentralized and message-driven, with no central authority. Since only the LLM-driven agent is wired to the mission controller, it is the sole agent able to issue proposals.

Beacon Communication: Each agent broadcasts its current position to all teammates as a `position_beacon` at a fixed interval, keeping coordination reliable even for teammates outside its direct sensory range.

Peer Communication: Simple agent-level commands (navigation goals, scoring rules, traversal penalties) follow a uniform two-step pattern: the injection is applied to the issuing agent locally, then packaged as a peer-injection message and broadcast to all teammates.

Commands that carry stronger team-synchronisation requirements (rendezvous and traffic control) instead use a two-phase commit protocol described below.

Two-Phase Commit Protocol: Commands that carry stronger team-synchronization requirements utilize a generalized two-phase commit (2PC) protocol. This ensures the team only commits to a synchronized interruption of their current tasks when it is collectively optimal to do so.

- **Phase 1: Propose.** The proposing agent broadcasts a *propose* message carrying a unique round identifier (`rid`) and the command parameters. Each teammate evaluates the proposal based on its current local state and replies with a *vote* message indicating its accept or reject decision for that `rid`.
- **Phase 2: Commit.** After a fixed collection window, the proposer checks for unanimity. If every expected teammate voted *accept*, the proposer broadcasts a *commit* message, and all agents (including the proposer) execute the command. If any vote is missing or negative, the proposer broadcasts *abort*, and the command is discarded globally.

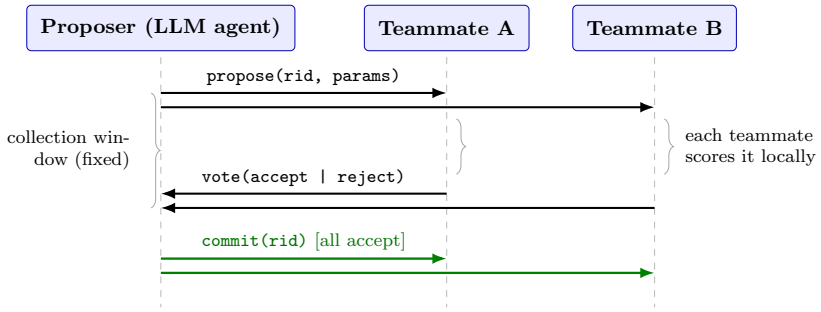

Figure 2: Two-phase commit for synchronized holds: the proposer broadcasts candidate hold tiles, each teammate scores them locally and votes, and the team commits only on unanimous acceptance within the collection window (otherwise it aborts globally).

Protocol Evaluation and Execution: We apply 2PC to the hold maneuvers `request_rendezvous` and `request_red_light`; a committed action injects a `HOLD_TILE` desire. A teammate votes by scoring the candidate hold tiles (around the meeting point for rendezvous; on alternate rows or columns for traffic control) with the reward-over-distance rule of Section 2.2, keeping the best `HOLD_TILE` score. To keep the comparison fair the carried value is first discounted by the parcel-reward decay expected during the wait, and the teammate accepts only if this discounted score outweighs its current desires.

Holds are then released by their command semantics: a rendezvous hold clears itself once position beacons confirm all teammates have gathered within the agreed radius, whereas a traffic-control hold persists until its time limit expires or the mission controller issues an explicit green-light command that releases the whole team at once.

4.4 Cooperation

Beyond reactive primitives, every k seconds the LLM agent opens a cooperation round, requesting teammates' beliefs and assembling a compact LLM context from three sources: a *roster* (each agent's position, load, and sensed enemies), a deterministic *geometry* summary of the spawn and delivery zones, and a *performance* record of each strategy's average per-round score over a sliding window. The prompt is steered with a few *few-shot examples* and *chain-of-thought* prompting, returning a structured rationale before committing to a strategy.

Strategies: Two cooperative strategies are available alongside a default non-cooperative baseline. In `ZONAL_RELAY`, one agent clears the spawn region and drops parcels at a fixed midpoint tile, while the other ferries them to delivery tiles. In `OPPORTUNISTIC`, agents forage independently and hand off parcels dynamically whenever a pickup occurs.

Roles: Each strategy assigns roles executed by a deterministic FSM that supersedes BDI deliberation, emitting a restricted set of state-gated desires. Spatial targets are computed from map geometry: the exchange tile is the reachable cell closest to the spawn–delivery midpoint, and each agent gets a distinct adjacent approach cell, injected as a high-reward `REACH_TILE` desire until the FSM transitions.

Handoff: Both strategies culminate in a handoff. In `ZONAL_RELAY` the exchange loops indefinitely; in `OPPORTUNISTIC` roles are fluid—the agent that acquires a parcel proposes a handoff via 2PC, becoming the *passer* once the partner commits as *receiver* (accepted only if the meeting tile is reachable). An explicit mission-controller directive does not alter this machinery; it merely adds a reward bonus to the handoff desire.